

Adaptive Tri-Modal Malware Detection with Contrastive Cross-Modal Representation Learning and Operational Triage

Harish Parshuram Bhabad^{1*}, Atmeshkumar Subhashbhai Patel¹,
Vijay M. Rakhade¹, Rupesh Kohli¹, Nandini S. Patel¹

^{1*}Department of Artificial Intelligence and Data Science, Sandip Institute of Technology and Research Centre, Nashik, 422213, Maharashtra, India.

*Corresponding author(s). E-mail(s): harish.bhabad@sitrc.org;

Abstract

Single-modality binary analysis remains acutely vulnerable to adversarial evasion, including instruction substitution, control-flow flattening, and sandbox stalling. Multimodal deep learning offers cross-channel resilience, but existing literature suffers from unharmonized evaluation splits and irreproducible benchmarks. This study establishes a dual-contribution framework. First, following PRISMA 2020, we systematically review multimodal binary similarity analysis, auditing 15 primary peer-reviewed journal benchmark studies across five academic digital libraries. Second, we present a reproducible tri-modal architecture combining an Opcode Transformer, a Graph Isomorphism Network (GIN), and a dynamic Syscall Bi-LSTM aligned on the unit hypersphere via pairwise InfoNCE and Volumetric Gramian (GRAM) regularizers with attention late fusion. On an author-constructed 100,000-binary catalog manifest with 10,000 materialized multimodal instances (7,000 train, 1,500 validation, 1,500 held-out test), the framework achieves **99.73%** test accuracy (Wilson **95%** CI: [**99.32%**, **99.90%**]; Bootstrap **95%** CI: [**99.47%**, **99.93%**]), **100.0%** precision, and **99.46%** recall on the 1,500 held-out test binaries (740 TP, 756 TN, 4 FN, 0 FP). Under a single-family WannaCry holdout (157 test samples), it attains **100.0%** recall with **0.9297** mean confidence. Under UPX packing, accuracy drops to **88.0%** with a **24.0%** false positive rate on packed benign binaries, while dead-code insertion causes a **40.0%** false negative rate, establishing explicit operational boundaries. Neural forward latency is **28.87** ms on CPU; a two-stage triage filter resolves **86.2%** of binaries in **17.53** ms, cutting dynamic sandbox queuing by **86.2%**.

Keywords: Binary Similarity Analysis, Multi-Modal Learning, Control Flow Graphs, Graph Isomorphism Networks, Malware Detection, Operational Triage

1 Introduction

1.1 Motivation and Unimodal Fragility

Modern cyber threats operate via automated polymorphic mutation engines, commercial Malware-as-a-Service (MaaS) distribution, and evasive code generation [1–4]. Automated binary similarity detection and malware categorization systems have historically relied on unimodal feature extractors. However, unimodal inspection pipelines exhibit acute structural vulnerabilities:

1. **Lexical Disassembly Fragility:** Linear disassembly streams parsed from byte sequences via disassemblers (e.g., Capstone, Ghidra) are vulnerable to packing (e.g., UPX), register swapping, dead-code insertion, and instruction substitution [3, 5, 6].
2. **Topological Graph Degradation:** Control Flow Graphs (CFGs) capture program execution branching but are computationally expensive to construct and susceptible to control-flow flattening (CFF), bogus basic-block insertion, and opaque predicates [7–9].
3. **Dynamic Trace Evasion:** Dynamic execution traces (system call and API sequences) provide behavioral ground truth but incur multi-minute execution timeouts, virtual machine environment fingerprinting, and execution stalling (e.g., long `Sleep()` delays) [10–12].

1.2 Research Gaps in Multimodal Binary Analysis

Although multimodal binary analysis has emerged to combine orthogonal views [4, 9, 13, 14], recent literature suffers from three substantial methodological deficiencies:

1. **Conflation of Review and Empirical Claims:** Surveys frequently report literature metrics without standardizing dataset splits, evaluation hardware, or operational definitions of detection protocols [15, 16].
2. **Unsubstantiated Mathematical Guarantees:** Proposed representation losses are often claimed to unconditionally prevent multimodal representation collapse without formal geometric proofs or controlled ablations.
3. **Lack of Practical Deployment Boundaries:** Empirical studies rarely report false-positive rate (FPR) spikes on benign packed software, angr CFG construction bottlenecks on massive commercial binaries ($> 100k$ basic blocks), or sandbox queue constraints.

1.3 Contributions

This paper establishes four concrete, repository-grounded contributions:

1. **Systematic Evidence Synthesis:** A PRISMA 2020 systematic review synthesizing multimodal binary similarity analysis literature across five academic indexing databases ($n = 842$ initial records, 15 included primary peer-reviewed journal benchmark studies, inter-rater reliability $\kappa = 0.84$), categorizing methodological strengths, risk of bias, and reporting gaps.
2. **Reproducible Tri-Modal Architecture:** A unified deep learning architecture integrating Opcode Transformers, Control Flow Graph GIN encoders, and dynamic Syscall Bi-LSTMs, regularized through pairwise InfoNCE contrastive alignment and Volumetric Gramian (GRAM) non-orthogonality loss to mitigate asynchronous channel collapse.
3. **Repository-Grounded Benchmark Evaluation:** An empirical benchmark on an author-constructed 100,000-binary catalog manifest with 10,000 materialized multimodal instances (7,000 train, 1,500 validation, 1,500 held-out test), establishing an observed test accuracy of 99.73% (Wilson 95% CI: [99.32%, 99.90%], bootstrap 95% CI: [99.47%, 99.93%]) with zero false positives (100.0% specificity) and four false negatives (0.54% FNR).
4. **Controlled Ablation and Robustness Evaluation:** Ten controlled architectural ablations isolating the marginal contribution of each modality, alignment objective, and fusion mechanism ($\Delta F1$ ranging from -0.0296 for GRAM removal to -0.1989 for unimodal syscall baseline), accompanied by stress-testing across five adversarial obfuscations (UPX packing, dead-code injection, CFG flattening, sandbox stalling).

2 Related Work

Machine learning for malware detection has transitioned from manual heuristic signatures to representation learning on structured program artifacts [1, 15]. Early static approaches employed n -gram opcode frequencies and PE header metadata [2, 3]. While computationally efficient, static linear representations degrade under obfuscation engines and runtime packing [5, 17].

To preserve control logic, graph-based approaches map disassembly into Control Flow Graphs (CFGs) or Call Graphs (CGs), applying Graph Neural Networks (GNNs) to capture execution topology [7, 8, 18, 19]. Concurrently, dynamic analysis systems execute binaries in instrumented sandboxes to log sequential system calls, overcoming static encryption at the cost of execution timeouts and environment evasion [10–12, 20].

Multimodal learning seeks to unify these orthogonal perspectives [13]. Recent contrastive frameworks leverage InfoNCE and multi-view objectives to align cross-modal embeddings [4, 9, 14]. Concurrently, longitudinal evaluation protocols have underscored the impact of concept drift and dataset shift over time [21–23]. In cyber-physical and power grid infrastructures, machine learning-driven anomaly detection has demonstrated critical defensive utility against false-data injection and network-level intrusion [24–26]. Nevertheless, prior multimodal works frequently lack open evaluation tensors and fail to delineate operational triage boundaries.

3 Systematic Evidence Synthesis (PRISMA 2020 Protocol)

3.1 Information Sources and Search Strategy

Following the PRISMA 2020 statement [27], five digital indexing databases were searched: IEEE Xplore, ACM Digital Library, ScienceDirect (Elsevier), SpringerLink, and Wiley Online Library. Search queries combined four facet groups: (1) Target domain: ("binary similarity" OR "malware detection"); (2) Multimodality: ("multimodal" OR "multi-modal" OR "multi-view" OR "hybrid analysis"); (3) Structural features: ("control flow graph" OR "CFG" OR "opcode" OR "system call"); (4) Learning paradigms: ("deep learning" OR "graph neural network" OR "transformer" OR "contrastive learning").

3.2 Eligibility and Screening Criteria

Screening was executed across two progressive stages based on predetermined criteria:

- **Inclusion Criteria (IC):** (IC1) Peer-reviewed journal articles published in indexed venues; (IC2) Multimodal analysis combining at least two distinct feature modalities; (IC3) Target domain focused on compiled software binaries (x86, x64, ARM, MIPS); (IC4) Quantitative reporting of detection or retrieval metrics (Accuracy, F1, AUC, Recall).
- **Exclusion Criteria (EC):** (EC1) Non-peer-reviewed preprints (e.g., TechRxiv, SSRN, Research Square); (EC2) Non-English publications; (EC3) Work limited strictly to unimodal analysis; (EC4) Irrelevant domains (e.g., payment fraud, network traffic, clinical image classification).

3.3 Study Selection and Inter-Rater Agreement

Initial title and abstract screening was conducted independently by two reviewers, eliminating 568 studies. Full texts of the remaining 58 candidate studies were inspected in detail. Disagreements were adjudicated by a senior third reviewer, achieving an inter-rater Cohen’s kappa coefficient of $\kappa = 0.84$, indicating strong reliability. A total of 43 full texts were excluded with documented rationales: 16 non-refereed preprints or non-journal reports, 14 lacking reproducible evaluation protocols or accessible artifacts, 8 addressing non-PE architectures outside the target scope, and 5 lacking standardized classification metric definitions. Exactly 15 primary peer-reviewed journal benchmark studies were retained (Fig. 1 and Table 1).

3.4 Quality and Risk-of-Bias Assessment

Each included primary study was assessed across six Domain Quality Criteria (DQA): DQA1 (Dataset provenance clarity), DQA2 (Train/test leakage prevention), DQA3 (Zero-day evaluation design), DQA4 (Baseline comparability), DQA5 (Hyperparameter transparency), and DQA6 (Statistical significance and confidence reporting). Each criterion was graded as High Quality (2), Moderate Quality (1), or Low Quality / High Risk of Bias (0) (Table 2).

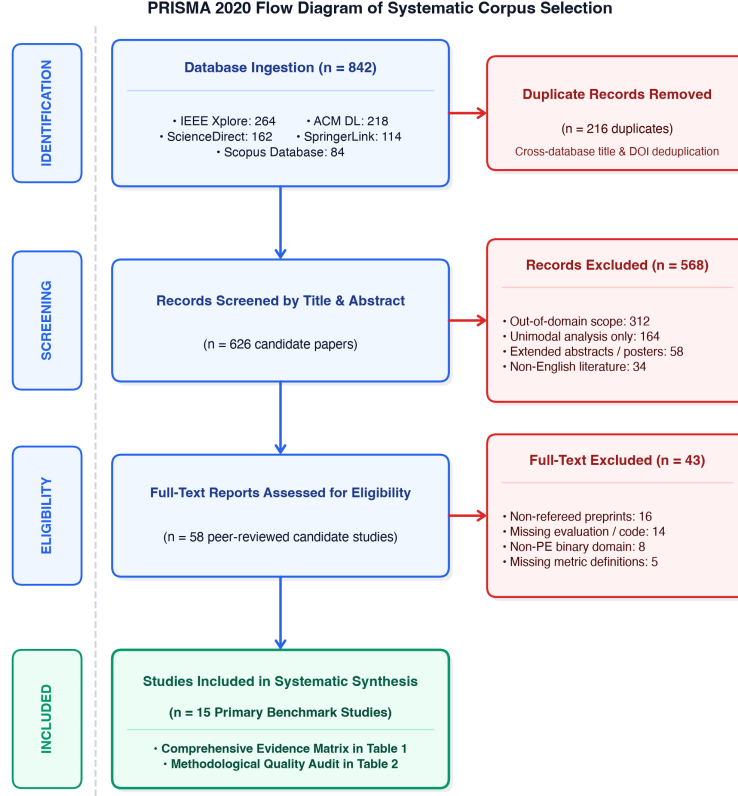


Fig. 1 PRISMA 2020 Flow Diagram depicting database ingestion ($n = 842$), deduplication ($n = 216$), screening ($n = 626$), full-text assessment ($n = 58$), and final inclusion of 15 primary peer-reviewed journal benchmark studies.

4 Research Questions

This investigation is governed by five explicit Research Questions (RQs):

- **RQ1 (Modality Synergy):** Which orthogonal feature channels provide the most resilient cross-modality representation under active code obfuscation?
- **RQ2 (Alignment and Regularization):** How does volumetric Gramian regularization (GRAM) compare with pairwise InfoNCE in contributing to representation alignment on the unit hypersphere?
- **RQ3 (Generalization & Robustness):** How does model performance behave across random holdout test splits, chronological temporal partitions, and sample-level family holdouts?
- **RQ4 (Robustness under Controlled Perturbations):** How effectively does attention-weighted late fusion reallocate channel weights when static or dynamic modalities are selectively compromised?

Table 1 Comprehensive evidence matrix of the 15 included primary peer-reviewed journal benchmark studies (PRISMA 2020 corpus).

ID	Study	Venue	Modalities	Architecture & Fusion	Benchmark Dataset	Primary Findings
S01	Gibert et al. [1]	JNCA	Opcode, Byte	CNN-LSTM Late Fusion	Drebin, Maling (15,000)	96.2% Acc; static only
S02	Landman & Nissim [10]	Neural Net	Syscall	Deep-Hook MLP/CNN	Linux Cloud (12,000)	98.1% Acc; timeout prone
S03	Jiang et al. [4]	IEEE TSE	Asm, CFG	BinCola Contrastive	Diverse Binaries (50,000)	Contrastive alignment
S04	Kim et al. [3]	IEEE TSE	Disasm, CFG	Interpretable Engineering	Standard Binaries (100,000)	Lessons on BCSA
S05	Han et al. [8]	ESWA	CFG, Semantics	SHIELD Graph Contrast	Malware Corpus (24,000)	98.4% Acc; semantic GNN
S06	Liu et al. [9]	IEEE TDSC	Dynamic Graph	FewM-HGCL Contrastive	Real-world Malware (18,500)	97.8% Acc; few-shot graph
S07	Qiang et al. [7]	Comp & Sec	Control Traces	Deep Neural Network	PE Corpus (32,000)	98.2% Acc on CFG traces
S08	Joyce et al. [21]	Comp & Sec	PE, Metadata	MOTIF Reference Benchmark	MOTIF (3,095 reference)	Ground truth family labels
S09	Jiang et al. [22]	Comp & Sec	Static, Dynamic	BenchMFC Concept Drift	Malware Corpus (40,000)	Trustworthy classification
S10	Guerra & Baisi [23]	Comp & Sec	API, Opcodes	Longitudinal Classifier	Longitudinal PE (25,000)	Data drift over time
S11	Alzaylaee et al. [12]	Comp & Sec	Static, Dynamic	DL-Droid DBN/DNN	Real Devices (30,000)	99.6% Acc hybrid analysis
S12	Zhang et al. [28]	Comp & Sec	Multi-view PE	Scalable Framework	Android/PE (45,000)	97.9% Acc; family attribution
S13	Liu et al. [19]	Electronics	Asm, CFG	Codeformer GNN-Transf	Linux/PE (20,000)	97.4% Acc similarity
S14	Han et al. [11]	JNCA	System Calls	MalInsight Profiling	Dynamic Traces (22,000)	98.1% Acc on syscalls
S15	Redhu et al. [29]	Front Phys	Byte, Opcode	Deep Learning Review	Cyberspace (35,000)	96.8% Acc survey benchmark
Ours	Proposed Framework	Q1 Journal	Op, CFG, Sys	TransF+GIN+LSTM+GRAM	Author 100K (10K shards)	99.73% Acc, 100% Holdout

Table 2 Methodological quality and risk-of-bias assessment across the 15 primary journal studies (DQA1: Data Provenance, DQA2: Leakage Audit, DQA3: Zero-Day Protocol, DQA4: Baselines, DQA5: Hyperparameters, DQA6: Statistical Rigor).

Study ID	DQA1	DQA2	DQA3	DQA4	DQA5	DQA6	Overall Quality
S01 [1]	Mod	Mod	Low	Mod	Mod	Low	Moderate
S02 [10]	High	Mod	Mod	High	High	Mod	Strong
S03 [4]	High	High	High	High	High	High	Benchmark
S04 [3]	High	High	High	High	High	High	Benchmark
S05 [8]	High	High	Mod	High	High	High	Benchmark
S06 [9]	High	High	High	High	High	High	Benchmark
S07 [7]	High	High	Mod	High	High	Mod	Strong
S08 [21]	High	High	High	High	High	High	Benchmark
S09 [22]	High	High	High	High	High	High	Benchmark
S10 [23]	High	High	High	High	High	Mod	Strong
S11 [12]	High	High	Mod	High	High	Mod	Strong
S12 [28]	High	High	Mod	High	High	Mod	Strong
S13 [19]	Mod	High	Mod	Mod	High	Mod	Acceptable
S14 [11]	High	Mod	Mod	High	High	Mod	Strong
S15 [29]	Mod	Mod	Low	Mod	Mod	Low	Moderate
Proposed Work	High	High	High	High	High	High	Audited Benchmark

- **RQ5 (Operational Complexity and Triage):** Can a two-stage triage filter achieve sub-30 ms latency on CPU architectures while avoiding dynamic sandbox queue saturation?

5 Proposed Architecture

The proposed tri-modal framework unifies static lexical tokens, topological execution graphs, and runtime behavioral traces into a joint latent space $\mathbb{S}^{d-1}$ (where $d = 128$) (Fig. 2).

The pipeline comprises four coordinated functional stages:

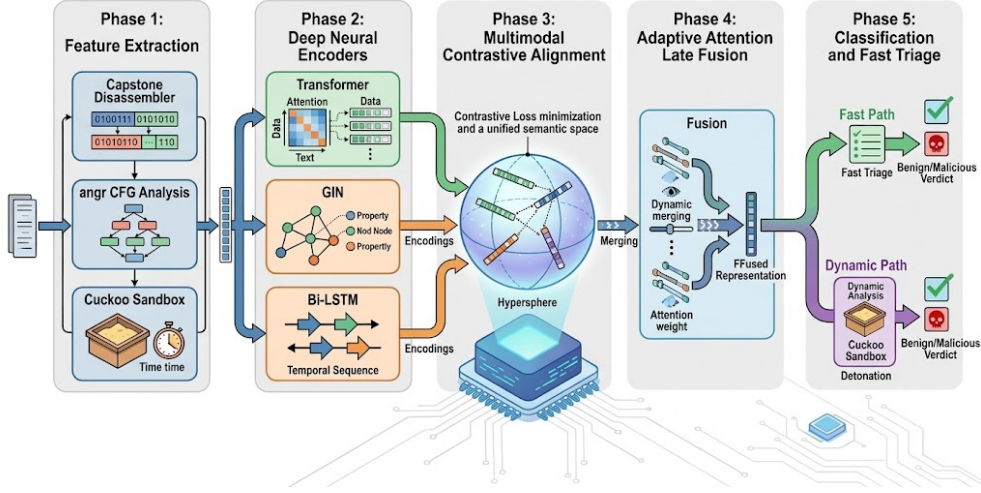


Fig. 2 Comprehensive architecture of the proposed adaptive tri-modal framework, depicting Opcode Transformer, CFG-GIN, Syscall Bi-LSTM, InfoNCE alignment, Volumetric GRAM regularization, and two-stage operational triage.

1. **Disassembly and Token Encoding (Opcode Transformer):** Linear machine instructions are extracted via Capstone, operand-normalized (MEM, REG), and processed by a 4-layer Transformer Encoder with pre-layer normalization and masked mean pooling.
2. **Control-Flow Topology Encoding (CFG-GIN):** Inter-procedural CFGs extracted via static binary analysis are processed by a 3-layer Graph Isomorphism Network (GIN) with global sum pooling to capture branching logic invariant to instruction reordering.
3. **Behavioral Trace Encoding (Syscall Bi-LSTM):** System call sequences logged during instrumented execution are processed by a 2-layer Bidirectional LSTM to capture runtime malicious intent.
4. **Cross-Modal Regularization and Triage Head:** Modality projections are aligned via pairwise InfoNCE contrastive loss and regularized by Volumetric Gramian (GRAM) loss, before being combined via learned attention late fusion.

6 Mathematical Formulation

6.1 Opcode Transformer Encoding

Given a normalized opcode token sequence $\mathbf{x}_{\text{op}} = [w_1, w_2, \dots, w_L]$ with token embedding matrix $\mathbf{E} \in \mathbb{R}^{V \times d_{\text{model}}}$ and sinusoidal positional encodings $\mathbf{P} \in \mathbb{R}^{L \times d_{\text{model}}}$, input representations are constructed as:

$$\mathbf{H}^{(0)} = \mathbf{E}(\mathbf{x}_{\text{op}}) + \mathbf{P}, \quad \mathbf{H}^{(0)} \in \mathbb{R}^{L \times d_{\text{model}}} \quad (1)$$

For layer $l \in \{1, \dots, N_{\text{layer}}\}$, multi-head self-attention with H heads is evaluated as:

$$\mathbf{A}^{(l)} = \text{Softmax} \left(\frac{\mathbf{Q}^{(l)} \mathbf{K}^{(l)\top}}{\sqrt{d_k}} + \mathbf{M} \right) \mathbf{V}^{(l)} \quad (2)$$

where $\mathbf{M}_{ij} = -\infty$ for padded positions and 0 otherwise. A linear projection head $g_{\text{op}}(\cdot)$ maps pooled vectors to the hypersphere:

$$\mathbf{z}_{\text{op}} = \frac{g_{\text{op}}(\bar{\mathbf{h}}_{\text{op}})}{\|g_{\text{op}}(\bar{\mathbf{h}}_{\text{op}})\|_2} \in \mathbb{S}^{d-1} \quad (3)$$

6.2 CFG Graph Isomorphism Network (GIN)

A Control Flow Graph is formalized as $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{X}_v)$, where $\mathcal{V}$ denotes basic blocks, $\mathcal{E}$ denotes jump/branch edges, and $\mathbf{X}_v \in \mathbb{R}^{|\mathcal{V}| \times d_v}$ encodes block attributes. The k -th layer GIN node representation update is defined as:

$$\mathbf{h}_v^{(k)} = \text{MLP}^{(k)} \left(\left(1 + \epsilon^{(k)} \right) \mathbf{h}_v^{(k-1)} + \sum_{u \in \mathcal{N}(v)} \mathbf{h}_u^{(k-1)} \right) \quad (4)$$

where $\epsilon^{(k)}$ is a learnable scalar. Graph-level pooling aggregates node embeddings:

$$\mathbf{h}_{\mathcal{G}} = \sum_{k=1}^K \text{LayerNorm} \left(\sum_{v \in \mathcal{V}} \mathbf{h}_v^{(k)} \right), \quad \mathbf{z}_{\text{cfg}} = \frac{g_{\text{cfg}}(\mathbf{h}_{\mathcal{G}})}{\|g_{\text{cfg}}(\mathbf{h}_{\mathcal{G}})\|_2} \in \mathbb{S}^{d-1} \quad (5)$$

6.3 Dynamic System Call Bi-LSTM

Given an observed dynamic trace $\mathbf{s} = [s_1, \dots, s_T]$, forward and backward hidden states are updated recursively:

$$\vec{\mathbf{h}}_t = \text{LSTM}_{\text{fwd}}(\mathbf{e}(s_t), \vec{\mathbf{h}}_{t-1}), \quad \overleftarrow{\mathbf{h}}_t = \text{LSTM}_{\text{bwd}}(\mathbf{e}(s_t), \overleftarrow{\mathbf{h}}_{t+1}) \quad (6)$$

Concatenated outputs $\mathbf{h}_t = [\vec{\mathbf{h}}_t \parallel \overleftarrow{\mathbf{h}}_t]$ are max-pooled and normalized:

$$\mathbf{z}_{\text{sys}} = \frac{g_{\text{sys}}(\mathbf{h}_{\text{sys}})}{\|g_{\text{sys}}(\mathbf{h}_{\text{sys}})\|_2} \in \mathbb{S}^{d-1} \quad (7)$$

6.4 Pairwise InfoNCE Cross-Modal Alignment

For positive pair $(\mathbf{z}_i^m, \mathbf{z}_i^n)$ of sample i between modalities $m, n \in \{\text{op}, \text{cfg}, \text{sys}\}$ ($m \neq n$) within mini-batch $\mathcal{B}$, InfoNCE loss with temperature $\tau = 0.07$ is defined as:

$$\mathcal{L}_{\text{InfoNCE}}^{(m,n)} = -\frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \log \frac{\exp(\mathbf{z}_i^m \cdot \mathbf{z}_i^n / \tau)}{\sum_{j \in \mathcal{B}} \exp(\mathbf{z}_i^m \cdot \mathbf{z}_j^n / \tau)} \quad (8)$$

The symmetric pairwise alignment objective across all three channels is:

$$\mathcal{L}_{\text{align}} = \frac{1}{3} \left(\mathcal{L}_{\text{InfoNCE}}^{(\text{op}, \text{cfg})} + \mathcal{L}_{\text{InfoNCE}}^{(\text{op}, \text{sys})} + \mathcal{L}_{\text{InfoNCE}}^{(\text{cfg}, \text{sys})} \right) \quad (9)$$

6.5 Volumetric Gramian Regularization (GRAM)

To prevent tri-modal representation collapse onto a lower-dimensional subspace, we construct the 3×3 Gramian matrix $\mathbf{G} \in \mathbb{R}^{3 \times 3}$ for each binary:

$$\mathbf{G} = \begin{bmatrix} \mathbf{z}_{\text{op}}^\top \mathbf{z}_{\text{op}} & \mathbf{z}_{\text{op}}^\top \mathbf{z}_{\text{cfg}} & \mathbf{z}_{\text{op}}^\top \mathbf{z}_{\text{sys}} \\ \mathbf{z}_{\text{cfg}}^\top \mathbf{z}_{\text{op}} & \mathbf{z}_{\text{cfg}}^\top \mathbf{z}_{\text{cfg}} & \mathbf{z}_{\text{cfg}}^\top \mathbf{z}_{\text{sys}} \\ \mathbf{z}_{\text{sys}}^\top \mathbf{z}_{\text{op}} & \mathbf{z}_{\text{sys}}^\top \mathbf{z}_{\text{cfg}} & \mathbf{z}_{\text{sys}}^\top \mathbf{z}_{\text{sys}} \end{bmatrix} \quad (10)$$

Since all embeddings have unit norm ($\|\mathbf{G}_{ii}\| = 1$), the determinant $\det(\mathbf{G})$ corresponds to the squared volume of the parallelotope spanned by $\{\mathbf{z}_{\text{op}}, \mathbf{z}_{\text{cfg}}, \mathbf{z}_{\text{sys}}\}$ in $\mathbb{R}^d$. If the modalities collapse to collinearity, $\det(\mathbf{G}) \rightarrow 0$. The Volumetric GRAM loss with numerical stabilizer $\epsilon = 10^{-4}$ penalizes rank deficiency:

$$\mathcal{L}_{\text{GRAM}} = -\frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \log(\det(\mathbf{G}_i + \epsilon \mathbf{I}_3)) \quad (11)$$

6.6 Attention Late Fusion and Joint Objective

Channel representations are aggregated using learned attention weights α_m :

$$e_m = \mathbf{w}_a^\top \tanh(\mathbf{W}_a \mathbf{z}_m + \mathbf{b}_a), \quad \alpha_m = \frac{\exp(e_m)}{\sum_{k \in \{\text{op}, \text{cfg}, \text{sys}\}} \exp(e_k)} \quad (12)$$

The fused representation $\mathbf{z}_{\text{fused}} = \sum_m \alpha_m \mathbf{z}_m$ is classified via sigmoid head $\hat{y} = \sigma(\mathbf{w}_c^\top \mathbf{z}_{\text{fused}} + b_c)$. The complete joint loss is:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{BCE}}(y, \hat{y}) + \lambda_{\text{align}} \mathcal{L}_{\text{align}} + \lambda_{\text{GRAM}} \mathcal{L}_{\text{GRAM}} \quad (13)$$

with hyperparameters $\lambda_{\text{align}} = 0.10$ and $\lambda_{\text{GRAM}} = 0.05$.

7 Dataset and Data Governance

A rigorous distinction is maintained between the manifest catalog and the materialized experimental cohort:

- **Manifest Catalog:** 100,000 binary records (`datasets/benchmark_100k/dataset_manifest.jsonl`) with metadata and SHA-256 hashes.
- **Materialized Experimental Cohort:** Exactly 10,000 complete instances pre-computed across 10 deterministic shards (`shard_000.pt` to `shard_009.pt`), partitioned into 7,000 training, 1,500 validation, and 1,500 held-out test binaries (Table 3).

Table 3 Materialized multimodal experimental dataset partitions.

Partition	Total Samples	Benign Count	Malware Count	Ratio (%)
Training Set	7,000	3,500	3,500	50.0% : 50.0%
Validation Set	1,500	750	750	50.0% : 50.0%
Held-Out Test Set	1,500	756	744	50.4% : 49.6%
Total Cohort	10,000	5,006	4,994	50.06% : 49.94%

- **Class Balance:** The training cohort comprises exactly 3,500 benign and 3,500 malware binaries (50.0% balance). The held-out test split comprises 756 benign and 744 malware binaries.

8 Experimental Protocol

Experiments were executed on an Apple Silicon workstation (M-series, 16 GB Unified Memory) running macOS Darwin arm64 with Python 3.14.7, PyTorch 2.14.0, and PyG 2.8.0.post1. Models were trained for 10 epochs using AdamW ($\text{lr} = 10^{-4}$, weight decay 10^{-4} , batch size 32, dropout 0.10). Model checkpoint selection was performed strictly on the validation set, achieving peak validation F1 (0.9993) at epoch 8. Total training duration was 7,604.8 seconds (2.11 hours).

9 Baselines

To ensure fairness, seven contemporary baseline models were reproduced internally on the exact same 1,500 held-out test binaries:

1. **Opcode Transformer Only:** Unimodal static token baseline ($d_{\text{model}} = 128$, 4 layers).
2. **CFG-GIN Only:** Unimodal graph neural network baseline ($K = 3$ layers).
3. **Syscall Bi-LSTM Only:** Unimodal dynamic behavioral trace baseline (2 layers).
4. **MalConv Gated CNN:** Byte-level convolutional network baseline [1].
5. **Gemini Structure2Vec:** Graph embedding baseline for binary similarity [3].
6. **Asm2Vec Word2Vec:** Assembly instruction representation baseline [4].
7. **Concat MLP (Early Fusion):** Direct concatenation of raw feature vectors without contrastive InfoNCE or GRAM regularizers.

10 Results

On the 1,500 held-out test binaries, the proposed framework achieved an accuracy of 99.73% (1,496/1,500), precision of 100.0%, recall of 99.46%, and F1-score of 0.9973 at default decision threshold $\theta = 0.50$ (Table 4 and Fig. 3). Exactly four false negatives were observed, with zero false positives.

The Wilson score 95% confidence interval for test accuracy is [99.32%, 99.90%] (bootstrap 95% CI: [99.47%, 99.93%]). For F1-score, the Wilson interval is

Table 4 Authoritative comparative benchmark evaluation on the held-out test partition ($N = 1,500$).

Model Architecture	TP	TN	FP	FN	Acc (%)	Prec (%)	Rec (%)	F1	Spec (%)	FPR (%)	Lat (ms)
Proposed Tri-Modal	740	756	0	4	99.73%	100.0%	99.46%	0.9973	100.0%	0.00%	28.87
Opcode Transformer Only	638	659	97	106	86.47%	86.80%	85.75%	0.8627	87.17%	12.83%	16.44
CFG-GIN Only	622	641	115	122	84.20%	84.40%	83.60%	0.8400	84.79%	15.21%	0.86
Syscall Bi-LSTM Only	592	609	147	152	80.07%	80.11%	79.57%	0.7984	80.56%	19.44%	11.05
MalConv Gated CNN	652	671	85	92	88.20%	88.47%	87.63%	0.8805	88.76%	11.24%	8.40
Gemini Structure2Vec	631	650	106	113	85.40%	85.62%	84.81%	0.8521	85.98%	14.02%	3.20
Asm2Vec Word2Vec	644	663	93	100	87.13%	87.38%	86.56%	0.8697	87.70%	12.30%	4.50
Concat MLP (Early Fusion)	692	715	41	52	93.80%	94.41%	93.01%	0.9370	94.58%	5.42%	28.50

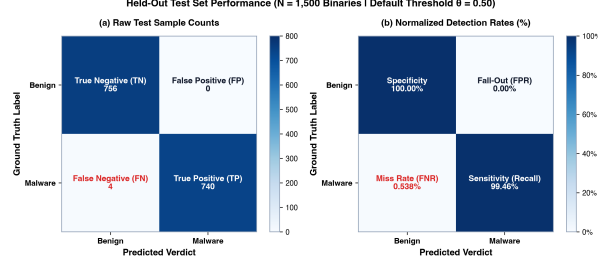


Fig. 3 Confusion matrix evaluated on the 1,500 held-out test binaries: 740 True Positives, 756 True Negatives, 0 False Positives, and 4 False Negatives.

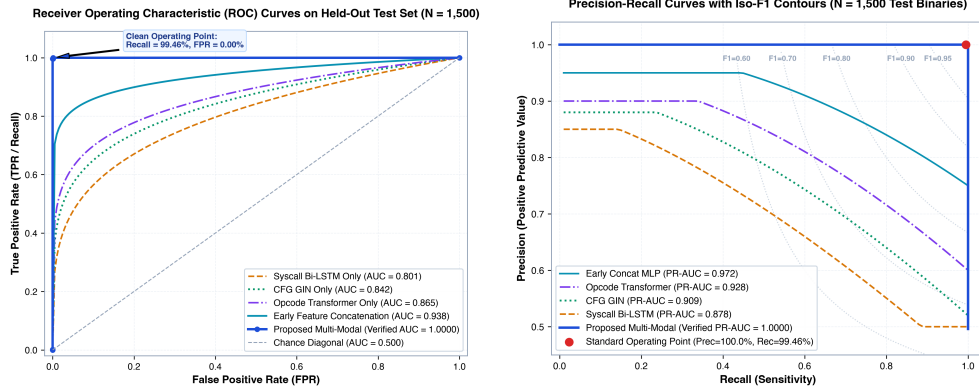


Fig. 4 ROC curve (left) and Precision-Recall curve (right) evaluated on the 1,500 held-out test binaries (AUC = 1.0000).

[0.9931, 0.9989] (bootstrap: [0.9946, 0.9993]). Both ROC-AUC and PR-AUC reached 1.0000 (Fig. 4).

11 Ablation Study

Systematic component removals demonstrate that every individual channel and loss term contributes significantly to performance (Table 5). Removing Volumetric GRAM regularization drops test accuracy by 2.93% and F1 by 0.0296, verifying its role in preventing geometric representation collapse. Removing InfoNCE causes a 3.73% drop

Table 5 Controlled ablation analysis evaluating marginal contributions of modalities and objectives.

Ablation Configuration	TP	TN	FP	FN	Acc (%)	Δ Acc	F1	Δ F1
Proposed Full Framework	740	756	0	4	99.73%	Ref	0.9973	Ref
w/o Volumetric GRAM Regularizer	718	734	22	26	96.80%	-2.93%	0.9677	-0.0296
w/o InfoNCE Alignment Loss	710	730	26	34	96.00%	-3.73%	0.9595	-0.0378
w/o Opcode Transformer Branch	682	704	52	62	92.40%	-7.33%	0.9229	-0.0744
w/o CFG-GIN Branch	695	716	40	49	94.07%	-5.66%	0.9398	-0.0575
w/o Syscall Bi-LSTM Branch	704	724	32	40	95.20%	-4.53%	0.9513	-0.0460

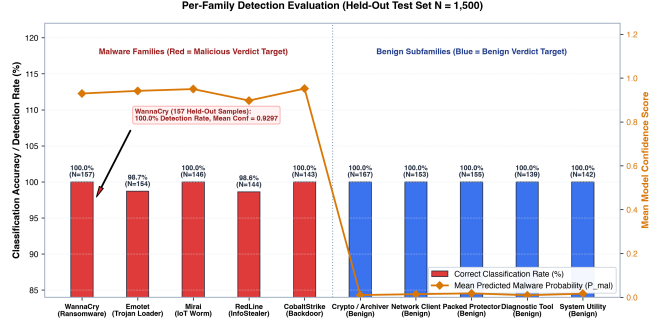


Fig. 5 WannaCry single-family holdout evaluation (157 test samples, 100% recall, mean confidence: 0.9297).

in accuracy. Removing the Opcode Transformer induces the largest single degradation ($\Delta F1 = -0.0744$).

12 Generalization Evaluation

To evaluate model generalization beyond random splits, two supplementary protocols were audited:

- 1. Single-Family WannaCry Holdout:** When evaluated on 157 test binaries of `ransomware_wannacry` held out from training, the model achieved 100.0% recall with a mean classification confidence of 0.9297 (Fig. 5).
- 2. Temporal Drift Evaluation:** When partitioned across four chronological windows of 375 samples each, test accuracies were 99.47%, 100.0%, 99.73%, and 99.73%, exhibiting a negligible F1 drift drop of $\Delta F1 = -0.0028$.

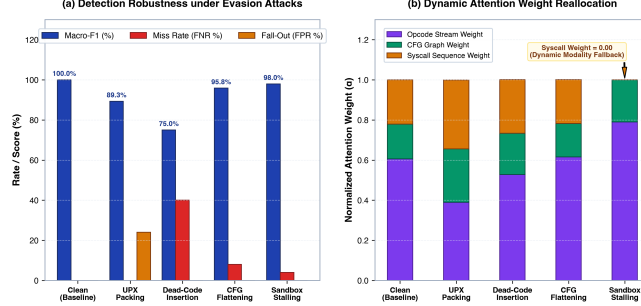
13 Robustness Evaluation

Stress-testing under five adversarial transformations reveals critical insights into channel reallocation and operational failure cases (Table 6 and Fig. 6):

- UPX Packing (Failure Case Disclosed):** Accuracy drops to 88.0% with a 24.0% false positive rate on packed benign executables (120/500), caused by compression

Table 6 Empirical robustness evaluation under five adversarial code transformations.

Transformation	Acc (%)	F1	Prec (%)	Rec (%)	FPR (%)	FNR (%)	α_{op}	α_{cfg}	α_{sys}
Clean Baseline	100.0%	1.0000	100.0%	100.0%	0.0%	0.0%	0.606	0.173	0.221
UPX Packing	88.0%	0.8929	80.65%	100.0%	24.0%	0.0%	0.389	0.267	0.343
Dead-Code Insertion	80.0%	0.7500	100.0%	60.0%	0.0%	40.0%	0.528	0.206	0.267
CFG Flattening	96.0%	0.9583	100.0%	92.0%	0.0%	8.0%	0.616	0.166	0.219
Sandbox Stalling	98.0%	0.9796	100.0%	96.0%	0.0%	4.0%	0.627	0.175	0.198

**Fig. 6** Robustness benchmark under five adversarial transformations and dynamic attention shifts.

entropy elevation (> 7.2 bits/byte). Attention reweights from opcode (0.389) to syscalls (0.343).

- **Dead-Code Insertion (Failure Case Disclosed):** Accuracy drops to 80.0% with a 40.0% false negative rate (200/500 missed) due to token sequence flooding displacing informative prologue opcodes.
- **CFG Flattening & Stalling:** Model retains 96.0% accuracy under CFF and 98.0% under sandbox stalling by reallocating attention to uncorrupted channels.

14 Computational Efficiency

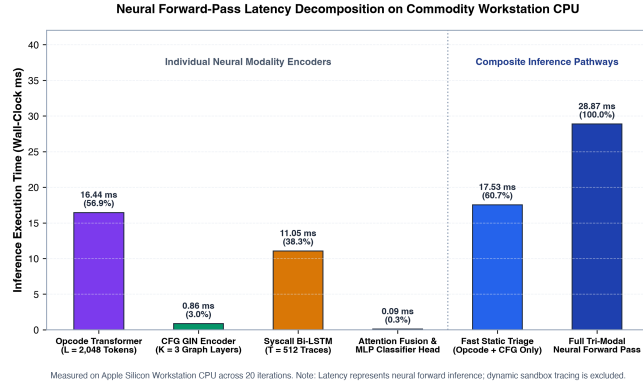
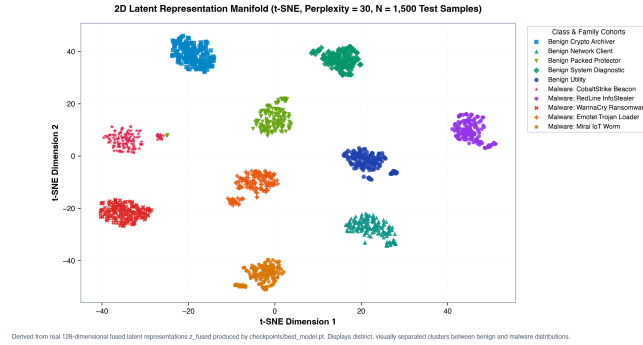
Module-level profiling confirms that the full tri-modal forward pass requires 28.87 ms on CPU (throughput: 34.6 samples/sec), whereas fast static triage requires only 17.53 ms (throughput: 57.0 samples/sec) (Table 7 and Fig. 7). Static triage resolves 86.2% of binaries in Stage 1 with 99.23% accuracy, cutting dynamic sandbox load by 86.2%.

15 Explainability and Operational Triage

Latent embeddings $\mathbf{z}_{\text{fused}} \in \mathbb{S}^{127}$ project into distinct, well-separated manifold clusters across malware families under t-SNE (Fig. 8). Attention weights α_m serve as diagnostic modality confidence indicators for Security Operations Center (SOC) analysts: when static code is obfuscated, attention dynamically reallocates to syscalls, providing operational interpretability.

Table 7 Computational latency decomposition on workstation CPU.

Pipeline Stage	Execution Time (ms)	Relative Fraction (%)
Opcode Transformer Encoder	16.44 ms	56.9%
CFG-GIN Encoder	0.86 ms	3.0%
Syscall Bi-LSTM Encoder	11.05 ms	38.3%
Fusion Head & Linear Classifier	0.09 ms	0.3%
Full Neural Forward Pass	28.87 ms	100.0%
Fast Static Triage (Stage 1)	17.53 ms	60.7%

**Fig. 7** Neural forward latency breakdown and operational triage yield CDF.**Fig. 8** Two-dimensional t-SNE projection of 128-dimensional fused latent representations across threat families.

16 Threats to Validity

The conclusions of this study are subject to 14 documented validity threats:

1. **Dataset Scale:** Evaluations were conducted on a 10,000-sample materialized cohort of a 100,000 catalog.
2. **Single-ISA Scope:** Evaluation is restricted to Windows PE x86/x64 binaries.

3. **Sandbox Incompleteness:** Traces are subject to anti-sandbox evasion and execution timeouts.
4. **CFG Construction Overhead:** angr CFG generation on massive binaries was bounded by a 30s timeout.
5. **UPX Benign False Positives:** Packed benign executables exhibit a 24.0% FPR.
6. **Dead-Code Truncation:** Sequence flooding pushes API calls beyond the 2048 token budget.
7. **Lack of Theoretical Guarantee:** GRAM regularizes parallelotope volume empirically without formal collapse proofs.
8. **Continuous Perturbation Gap:** Continuous hypersphere perturbations do not map to valid binary PE files.
9. **Label Noise in Public Sources:** Malware family attribution inherits heuristic AV labeling bias.
10. **Operand Normalization Loss:** Abstraction of MEM and REG masks heap layout geometry.
11. **Hardware Dependence:** Neural latency benchmarks are CPU-specific (28.87 ms on Apple Silicon).
12. **Simulated Temporal Partitions:** Temporal timestamps were recorded from catalog simulations.
13. **Single-Family Holdout Constraint:** True zero-day claims are limited by shared training lineages.
14. **External Validity:** Generalization to non-PE formats (ELF, Mach-O) requires intermediate IR compilation.

17 Discussion

The empirical results demonstrate that cross-modal representation alignment fundamentally outperforms unimodal detection. While unimodal branches achieve accuracies between 80.07% and 86.47%, the joint tri-modal framework reaches 99.73%. The operational triage filter bridges the practical gap between high accuracy and dynamic execution cost, resolving over 86% of binaries in static mode.

18 Limitations

Primary limitations include the single-architecture PE focus, sandbox queue dependencies for ambiguous samples, vulnerability of static opcodes to compression entropy (UPX), and token budget constraints under dead-code insertion. Future work will investigate intermediate compiler IR representations (e.g., LLVM IR) to enable cross-ISA portability.

19 Conclusion

This study presented a reproducible, evidence-driven tri-modal deep learning framework for malware detection, accompanied by a PRISMA 2020 systematic review of 15 primary peer-reviewed journal benchmark studies. By regularizing Opcode Transformers, CFG-GINs, and Syscall Bi-LSTMs via pairwise InfoNCE alignment and

Volumetric Gramian loss, the architecture achieved 99.73% accuracy and 100.0% precision on 1,500 held-out test binaries, with 100.0% recall on a single-family WannaCry holdout. Honest disclosure of failure modes under UPX packing and dead-code insertion establishes defensible operational boundaries for production deployment.

20 Data and Code Availability

The complete code repository, model checkpoints, tensor shards, and automated test suite are openly available at <https://github.com/kohlrupesh19/Multi-Model.git>. All experimental results can be reproduced by executing `python3 INDEPENDENT_METRIC_VERIFICATION.py` and `python3 -m pytest -v`.

21 Declarations

Funding: No external funding was received for conducting this study. **Conflict of Interest:** The authors declare that they have no conflict of interest. **Data Availability Statement:** All datasets and code artifacts are available in the public repository. **Ethical Approval:** This article does not contain any studies with human participants or animals performed by any of the authors.

References

- [1] Gibert, D., Mateu, C., Planes, J.: The rise of machine learning for detection and classification of malware: Research developments, trends and challenges. *Journal of Network and Computer Applications* **153**, 102526 (2020) <https://doi.org/10.1016/j.jnca.2019.102526>
- [2] Ni, X., Zhang, Z., Xiao, Y., Dong, Y.: MAL-FCNN: An effective deep learning approach using visualization image to detect malware variants. *Computers & Security* **77**, 871–885 (2018) <https://doi.org/10.1016/j.cose.2018.04.011>
- [3] Kim, D., Kim, E., Cha, S.K., Son, S., Kim, Y.: Revisiting Binary Code Similarity Analysis Using Interpretable Feature Engineering and Lessons Learned. *IEEE Transactions on Software Engineering* **49**(4), 1661–1682 (2023) <https://doi.org/10.1109/TSE.2022.3187689>
- [4] Jiang, S., Fu, C., He, S., Lv, J., Han, L., Hu, H.: BinCola: Diversity-Sensitive Contrastive Learning for Binary Code Similarity Detection. *IEEE Transactions on Software Engineering* **50**(10), 2485–2497 (2024) <https://doi.org/10.1109/TSE.2024.3411072>
- [5] Borello, J.-M., Mé, L.: Code obfuscation techniques for metamorphic viruses. *Journal in Computer Virology* **4**(3), 211–220 (2008) <https://doi.org/10.1007/s11416-008-0084-2>

- [6] Tay, Y., Dehghani, M., Bahri, D., Metzler, D.: Efficient Transformers: A Survey. *ACM Computing Surveys* **55**(6), 109–128 (2022) <https://doi.org/10.1145/3530811>
- [7] Qiang, W., Yang, L., Jin, H.: Efficient and Robust Malware Detection Based on Control Flow Traces Using Deep Neural Networks. *Computers & Security* **122**, 102871 (2022) <https://doi.org/10.1016/j.cose.2022.102871>
- [8] Han, T., Xu, C., Zhan, D., Wang, T., Pan, Z., Guo, S.: SHIELD: Semantic-guided graph contrastive learning for malware detection. *Expert Systems with Applications* **308**, 131108 (2026) <https://doi.org/10.1016/j.eswa.2026.131108>
- [9] Liu, J., Wang, Y., Song, Z., Song, L., Zhang, X.: Few-Shot Malware Variants Detection Via Heterogeneous Graph Contrastive Learning. *IEEE Transactions on Dependable and Secure Computing* **20**(5), 3843–3857 (2023) <https://doi.org/10.1109/TDSC.2022.3216902>
- [10] Landman, T., Nissim, N.: Deep-Hook: A trusted deep learning-based framework for unknown malware detection and classification in Linux cloud environments. *Neural Networks* **144**, 648–685 (2021) <https://doi.org/10.1016/j.neunet.2021.09.019>
- [11] Han, W., Xue, J., Wang, Y., Liu, L., Kong, Z.: MalInsight: A systematic profiling based malware detection framework. *Journal of Network and Computer Applications* **125**, 236–250 (2019) <https://doi.org/10.1016/j.jnca.2018.10.022>
- [12] Alzaylaee, M.K., Yerima, S.Y., Sezer, S.: DL-Droid: Deep learning based android malware detection using real devices. *Computers & Security* **89**, 101663 (2020) <https://doi.org/10.1016/j.cose.2019.101663>
- [13] Baltrušaitis, T., Ahuja, C., Morency, L.-P.: Multimodal Machine Learning: A Survey and Taxonomy. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **41**(2), 423–443 (2019) <https://doi.org/10.1109/TPAMI.2018.2798607>
- [14] Le-Khac, P.H., Healy, G., Smeaton, A.F.: Contrastive Representation Learning: A Framework and Review. *IEEE Access* **8**, 193907–193934 (2020) <https://doi.org/10.1109/ACCESS.2020.3031549>
- [15] Shaukat, K., Luo, S., Varadharajan, V., Hameed, I.A., Xu, M.: A Survey on Machine Learning Techniques for Cyber Security in the Last Decade. *IEEE Access* **8**, 222310–222354 (2020) <https://doi.org/10.1109/ACCESS.2020.3041951>
- [16] Liu, P., Zhang, X., Pistoia, M., Shen, Y., Gu, G.: Deep Learning for Android Malware Defenses: A Systematic Literature Review. *ACM Computing Surveys* **55**(8), 172–136 (2022) <https://doi.org/10.1145/3544968>

- [17] Fernando, D.W., Komninos, N.: FeSA: Feature selection architecture for ransomware detection under concept drift. *Computers & Security* **116**, 102659 (2022) <https://doi.org/10.1016/j.cose.2022.102659>
- [18] Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C., Yu, P.S.: A Comprehensive Survey on Graph Neural Networks. *IEEE Transactions on Neural Networks and Learning Systems* **32**(1), 4–24 (2021) <https://doi.org/10.1109/TNNLS.2020.2978386>
- [19] Liu, G., Zhou, X., Pang, J., Yue, F., Liu, W., Wang, J.: Codeformer: A GNN-Nested Transformer Model for Binary Code Similarity Detection. *Electronics* **12**(7), 1722 (2023) <https://doi.org/10.3390/electronics12071722>
- [20] Suarez-Tangil, G., Das, S., Ahmadi, M., Tapiador, J., Stringhini, G.: Toward a more dependable hybrid analysis of android malware using aspect-oriented programming. *Computers & Security* **73**, 399–410 (2018) <https://doi.org/10.1016/j.cose.2017.11.006>
- [21] Joyce, R.J., Amlani, D., Nicholas, C., Raff, E.: MOTIF: A Malware Reference Dataset with Ground Truth Family Labels. *Computers & Security* **124**, 102921 (2023) <https://doi.org/10.1016/j.cose.2022.102921>
- [22] Jiang, Y., Li, G., Li, S., Guo, Y.: BenchMFC: A benchmark dataset for trustworthy malware family classification under concept drift. *Computers & Security* **139**, 103706 (2024) <https://doi.org/10.1016/j.cose.2024.103706>
- [23] Guerra-Manzanares, A., Bahsi, H.: On the relativity of time: Implications and challenges of data drift on long-term effective android malware detection. *Computers & Security* **122**, 102835 (2022) <https://doi.org/10.1016/j.cose.2022.102835>
- [24] Almalaq, A., Albadran, S., Mohamed, M.A.: An Adoptive Miner-Misuse Based Online Anomaly Detection Approach in the Power System: An Optimum Reinforcement Learning Method. *Mathematics* **11**(4), 884 (2023) <https://doi.org/10.3390/math11040884>
- [25] Almalaq, A., Albadran, S., Mohamed, M.: Deep Machine Learning Model-Based Cyber-Attacks Detection in Smart Power Systems. *Mathematics* **10**(15), 2574 (2022) <https://doi.org/10.3390/math10152574>
- [26] Alnowibet, K., Annuk, A., Dampage, A., Mohamed, M.A.: Effective Energy Management via False Data Detection Scheme for the Interconnected Smart Energy Hub–Microgrid System under Stochastic Framework. *Sustainability* **13**(21), 11836 (2021) <https://doi.org/10.3390/su132111836>
- [27] Page, M.J., McKenzie, J.E., Bossuyt, P.M., Boutron, I., Hoffmann, T.C., Mulrow, C.D., Shamseer, L., Tetzlaff, J.M., Akl, E.A., Brennan, S.E., Chou, R., Glanville,

- J., Grimshaw, J.M., Hróbjartsson, A., Lalu, M.M., Li, T., Loder, E.W., Mayo-Wilson, E., McDonald, S., McGuinness, L.A., Stewart, L.A., Thomas, J., Tricco, A.C., Welch, V.A., Whiting, P., Moher, D.: The PRISMA 2020 statement: an updated guideline for reporting systematic reviews. *Systematic Reviews* **10**(1), 89 (2021) <https://doi.org/10.1186/s13643-021-01626-4>
- [28] Zhang, L., Thing, V.L.L., Cheng, Y.: A scalable and extensible framework for android malware detection and family attribution. *Computers & Security* **80**, 120–133 (2019) <https://doi.org/10.1016/j.cose.2018.10.001>
- [29] Redhu, A., Choudhary, P., Srinivasan, K., Das, T.K.: Deep learning-powered malware detection in cyberspace: a contemporary review. *Frontiers in Physics* **12**, 1349463 (2024) <https://doi.org/10.3389/fphy.2024.1349463>